

数值计算方法 第二次平时作业

自动化（控制）

1 问题描述

降落伞问题中，实际的空气阻力通常与下降速度的平方成正比，即 $F_U = -cv^2$ ，则降落伞运动的微分方程为

$$\frac{dv}{dt} = g - \frac{c}{m}v^2$$

若初始时处于静止状态（ $t = 0$ 时， $v = 0$ ），可得：

$$v(t) = \sqrt{\frac{gm}{c}} \tanh\left(\sqrt{\frac{gc}{m}} t\right)$$

其中 $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ 。

假设降落伞的质量为 $m = 68.1 \text{ kg}$ ，重力加速度为 $g = 9.81 \text{ m/s}^2$ 。要求降落伞自由落体运动 $t = 10 \text{ s}$ 后速率到 30 m/s ，分别用二分法、试位法、不动点迭代、Newton-Raphson 法和割线法求解降落伞的阻力系数 c ，并对各种方法进行比较。

2 问题分析

将已知参数代入速度公式并令 $v(10) = 30$ ，可得关于 c 的非线性方程：

$$f(c) = \sqrt{\frac{gm}{c}} \tanh\left(\sqrt{\frac{gc}{m}} \cdot t\right) - 30 = 0$$

该方程无法用解析方法直接求解，因此需要采用数值方法求根。由物理分析可知：当 c 很小时阻力几乎为零， $v(10)$ 接近自由落体速度 $g \times t = 98.1 \text{ m/s}$ ，远大于 30 ，此时 $f(c) > 0$ ；当 c 很大时阻力很大， $v(10)$ 远小于 30 ，此时 $f(c) < 0$ 。因此 $f(c)$ 在正半轴上存在唯一零点。

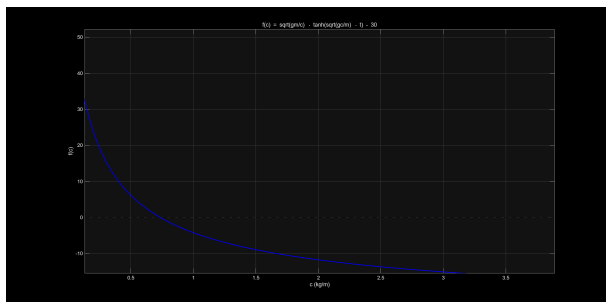


图 1: $f(c)$ 的函数图像

在 MATLAB 中绘制出 $f(c)$ 的图像如图 1 所示。

由此可知根大约在 $c \approx 0.74$ 附近。因此在求解的过程中，我们可以设置求根区间为 $[0.1, 50]$ 来得到根。

3 二分法

3.1 算法分析

要使用二分法进行求根，我们必须要先找到一个包含根的区间，将区间二等分后通过判断 $f(x)$ 的符号，逐步将有根区间缩小，直至有根区间足够地小，便可求出满足精度要求的近似根。注意到

$$f(0.1) > 0, \quad f(50) < 0$$

因此我们可以选取区间 $[0.1, 50]$ 进行二分法求根。

二分法求根的实现步骤如下：

1. 猜测初始下界 x_l 和上界 x_u ，使 $f(x_l)f(x_u) < 0$ ；
2. 计算中间点 $x_r = \frac{x_l + x_u}{2}$ ；
3. 根据情况确定根所在的区间：如果 $f(x_l)f(x_r) < 0$ ，则根落在左边子区间，取 $x_u = x_r$ ；如果 $f(x_l)f(x_r) > 0$ ，则根落在右边子区间，取 $x_l = x_r$ ；如果 $f(x_l)f(x_r) = 0$ ，则 x_r 即为所要求的根；
4. 终止条件：当 $|f(x_r)| < \varepsilon$ 或区间宽度 $(x_u - x_l)/2 < \varepsilon$ 时停止迭代。

3.2 MATLAB 程序

程序中首先设置了 $f(c)$ 以及相应参数。容差设置为 10^{-8} ，最大迭代次数设置为 100 次。完整 MATLAB 代码附在报告最后，此处仅展示二分法求根的部分。

```
function [root, iter, history] = my_bisection(f, a, b, tol, max_iter)
    history = [];
    for iter = 1:max_iter
        c = (a + b) / 2;
        history = [history; c];
        if abs(f(c)) < tol || (b - a)/2 < tol
            root = c; return;
        end
        if f(a) * f(c) < 0
            b = c;
        else
            a = c;
        end
    end
    root = (a + b) / 2;
end
```

3.3 运行结果及分析

程序运行结果中与二分法相关的输出如下：

二分法: $c = 0.73793065$, 迭代次数 = 33, $f(c) = -1.29e-09$

在给定区间 $[0.1, 50]$ 中，二分法经过 33 次迭代找到了根 $c \approx 0.73793065$ 。二分法的收敛速度为线性收敛，每次迭代区间减半，因此达到精度 ε 需要约 $\log_2 \frac{b-a}{\varepsilon}$ 次迭代。对于本题， $\log_2 \frac{49.9}{10^{-8}} \approx 32.2$ ，与实际迭代次数 33 一致。二分法虽然收敛速度最慢，但其优势在于只要初始区间包含根，就一定能收敛，稳定性最强。

4 试位法

4.1 算法分析

试位法的基本思想是通过一条直线连接 $[x_l, f(x_l)]$ 和 $[x_u, f(x_u)]$ ，直线与 x 轴的交点作为新的根的估计值：

$$x_r = x_u - \frac{f(x_u)(x_l - x_u)}{f(x_l) - f(x_u)}$$

然而，单纯的试位法存在一个显著问题：当函数曲率较大时，一个划界点可能保持不动，导致很差的收敛性。在本题的初步测试中，使用标准试位法需要超过 50 次迭代才能收敛，远慢于预期。分析发现，由于 $f(c)$ 在根附近曲率较大，区间左端点 a 长期固定不变，使得线性插值偏离实际根的位置，收敛效率低下。

为解决这一问题，我们采用了修正试位法（Illinois 方法）进行改进：

1. 使用计数器检测一个边界是否连续两次迭代保持不变；
2. 如果出现这种情况，将停滞的边界点处的函数值变为原来的一半，迫使插值直线向根的方向偏移。

这种修正有效打破了端点“粘住”的僵局，大幅提升了收敛速度。

4.2 MATLAB 程序

程序中首先设置了 $f(c)$ 以及相应参数。容差设置为 10^{-8} ，最大迭代次数设置为 100 次。完整 MATLAB 代码附在报告最后，此处仅展示修正试位法求根的部分。

```
function [root, iter, history] = my_regula_falsi(f, a, b, tol, max_iter)
    fa = f(a); fb = f(b);
    history = []; stag_a = 0; stag_b = 0;
    for iter = 1:max_iter
        c = b - fb * (b - a) / (fb - fa);
        fc = f(c);
```

```

history = [history; c];
if abs(fc) < tol, root = c; return; end
if fa * fc < 0
    b = c; fb = fc; stag_b = 0;
    stag_a = stag_a + 1;
    if stag_a >= 2, fa = fa / 2; end % 修正停滞端
else
    a = c; fa = fc; stag_a = 0;
    stag_b = stag_b + 1;
    if stag_b >= 2, fb = fb / 2; end
end
end
root = c;
end

```

4.3 运行结果及分析

程序运行结果中与试位法相关的输出如下：

试位法(修正): $c = 0.73793065$, 迭代次数 = 10, $f(c) = -2.36e-12$

修正试位法仅用 10 次迭代就收敛到了与二分法相同的根，且残差 $|f(c)| = 2.36 \times 10^{-12}$ 比二分法更小。相比之下，未修正的标准试位法需要超过 50 次迭代，修正机制将迭代次数减少了约 80%。与二分法相比，试位法利用了函数值的大小信息进行线性插值，因此通常能更快地逼近根；但在函数曲率较大的区域，必须配合修正机制才能发挥其优势。

5 不动点迭代

5.1 算法分析

运用不动点迭代要求将方程写成 $c = g(c)$ 的形式。对于本题，将速度公式变形可得：

$$v_{\text{target}} = \sqrt{\frac{gm}{c}} \tanh\left(\sqrt{\frac{gc}{m}} \cdot t\right) \Rightarrow c = \frac{gm}{v_{\text{target}}^2} \tanh^2\left(\sqrt{\frac{gc}{m}} \cdot t\right)$$

因此迭代函数为

$$g(c) = \frac{gm}{v_{\text{target}}^2} \tanh^2\left(\sqrt{\frac{gc}{m}} \cdot t\right)$$

由不动点定理，如果在根附近 $|g'(c)| < 1$ ，则迭代 $c_{n+1} = g(c_n)$ 将收敛到唯一的不动点。需要注意的是，不动点迭代的收敛性高度依赖于 $g(c)$ 的选取——不同的变形可能导致完全不同的收敛表现，甚至发散。

5.2 MATLAB 程序

程序中首先设置了 $f(c)$ 以及相应参数。容差设置为 10^{-8} ，最大迭代次数设置为 100 次。完整 MATLAB 代码附在报告最后，此处仅展示不动点迭代法求根的部分。

```
function [root, iter, history] = my_fixed_point(g, x0, tol, max_iter)
    x = x0; history = [x];
    for iter = 1:max_iter
        x_new = g(x);
        history = [history; x_new];
        if abs(x_new - x) < tol, root = x_new; return; end
        if abs(x_new) > 1e10 || isnan(x_new) || ~isreal(x_new)
            warning('不动点迭代发散。');
            root = x_new; return;
        end
        x = x_new;
    end
    root = x;
end
```

5.3 运行结果及分析

程序运行结果中与不动点迭代法相关的输出如下：

```
不动点迭代: c = 0.73793065, 迭代次数 = 6, f(c) = -1.48e-10
```

不动点迭代仅需 6 次就收敛，速度很快。这是因为所选迭代函数 $g(c)$ 在根附近的导数 $|g'(c)|$ 远小于 1，收敛因子有利。不动点迭代的实现非常简单，不涉及导数计算，计算量较小；但如果选择了不合适的迭代函数（使得 $|g'(c)| > 1$ ），则迭代将发散。

6 Newton-Raphson 方法

6.1 算法分析

Newton-Raphson 法的基本思想是从一个初始猜测值开始，通过线性近似来快速逼近方程的根。迭代公式为

$$c_{n+1} = c_n - \frac{f(c_n)}{f'(c_n)}$$

对于本题， $f(c)$ 的导数为（令 $\alpha = \sqrt{gm}$ ， $\beta = t\sqrt{g/m}$ ）：

$$f'(c) = \alpha \left[-\frac{1}{2c^{3/2}} \tanh(\beta\sqrt{c}) + \frac{\beta}{2c} \cdot \operatorname{sech}^2(\beta\sqrt{c}) \right]$$

程序中同时实现了解析导数和数值导数（中心差分 $f'(c) \approx \frac{f(c+h) - f(c-h)}{2h}$ ），并验

证两者一致性：在 $c = 0.5$ 处差异为 6.61×10^{-8} ，在 $c = 1.0$ 处差异为 1.16×10^{-7} ，确认了解析导数公式的正确性。

6.2 MATLAB 程序

程序中首先设置了 $f(c)$ 以及相应参数。容差设置为 10^{-8} ，最大迭代次数设置为 100 次。完整 MATLAB 代码附在报告最后，此处仅展示 Newton-Raphson 法求根的部分。

```
function [root, iter, history] = my_newton(f, df, x0, tol, max_iter)
    x = x0; history = [x];
    for iter = 1:max_iter
        f_val = f(x); df_val = df(x);
        if abs(df_val) < 1e-15
            warning('Newton: 导数接近零。');
            root = x; return;
        end
        x_new = x - f_val / df_val;
        history = [history; x_new];
        if x_new <= 0 % 防止迭代到负数区域
            x_new = x / 2;
            history(end) = x_new;
        end
        if abs(x_new - x) < tol, root = x_new; return; end
        x = x_new;
    end
    root = x;
end
```

6.3 运行结果及分析

程序运行结果中与 Newton-Raphson 法相关的输出如下：

```
Newton法(解析导数): c = 0.73793065, 迭代次数 = 5, f(c) = 3.55e-15
Newton法(数值导数): c = 0.73793065, 迭代次数 = 5, f(c) = 0.00e+00
```

Newton-Raphson 法仅需 5 次迭代即达到机器精度级别 ($|f(c)| \approx 10^{-15}$)，体现了二次收敛的优势。解析导数与数值导数的结果完全一致（迭代次数相同、精度相当），说明对于本题，数值微分是解析导数的良好替代。

Newton-Raphson 法的优点是收敛速度快，但也有局限性：如果初始猜测值不够接近真实根，迭代可能发散（如跳到负数区域）；当 $f'(c)$ 接近零时，更新步长过大，也可能导致失效。因此程序中加入了防护机制，当迭代值为负时自动回退到当前值的一半。

7 割线法

7.1 算法分析

割线法可以被看作是 Newton-Raphson 法的一种变体。割线法不需要计算函数的导数，而是使用两个近似点上的函数值来估计导数。迭代公式为：

$$c_{n+1} = c_n - f(c_n) \cdot \frac{c_n - c_{n-1}}{f(c_n) - f(c_{n-1})}$$

割线法的收敛阶约为 1.618（黄金比例），介于线性收敛和二次收敛之间。虽然收敛阶略低于 Newton 法，但每步不要求导数，综合效率很有竞争力。

7.2 MATLAB 代码

程序中首先设置了 $f(c)$ 以及相应参数。容差设置为 10^{-8} ，最大迭代次数设置为 100 次。完整 MATLAB 代码附在报告最后，此处仅展示割线法求根的部分。

```
function [root, iter, history] = my_secant(f, x0, x1, tol, max_iter)
    history = [x0; x1];
    for iter = 1:max_iter
        f0 = f(x0); f1 = f(x1);
        if abs(f1 - f0) < 1e-15
            warning('割线法：分母接近零。');
            root = x1; return;
        end
        x2 = x1 - f1 * (x1 - x0) / (f1 - f0);
        history = [history; x2];
        if x2 <= 0, x2 = x1 / 2; end % 防止跳到负数
        if abs(x2 - x1) < tol, root = x2; return; end
        x0 = x1; x1 = x2;
    end
    root = x1;
end
```

7.3 运行结果及分析

程序运行结果中与割线法相关的输出如下：

```
割线法：c = 0.73793065, 迭代次数 = 7, f(c) = 3.55e-15
```

割线法用 7 次迭代达到了与 Newton-Raphson 法相同的精度 ($|f(c)| = 3.55 \times 10^{-15}$)。相比于 Newton 法，割线法不需要计算导数，每步只需一次函数求值，在代码实现上更为简洁。虽然收敛阶 (≈ 1.618) 略低于 Newton 法的二次收敛，但综合考虑每步的计算代价，割线法的效率非常有竞争力。

8 比较总结

综合上述分析，五种方法均收敛到 $c \approx 0.73793065 \text{ kg/m}$ ，但在收敛速度、稳定性和使用条件上存在显著差异。各方法收敛过程的对比如图 2 所示。

方法	迭代次数	$ f(c) $	收敛阶	是否需要导数
二分法	33	1.29×10^{-9}	1（线性）	否
试位法（修正）	10	2.36×10^{-12}	1~1.618	否
不动点迭代	6	1.48×10^{-10}	1（线性）	否
Newton-Raphson	5	3.55×10^{-15}	2（平方）	是
割线法	7	3.55×10^{-15}	≈ 1.618	否

表 1: 五种方法求解结果对比

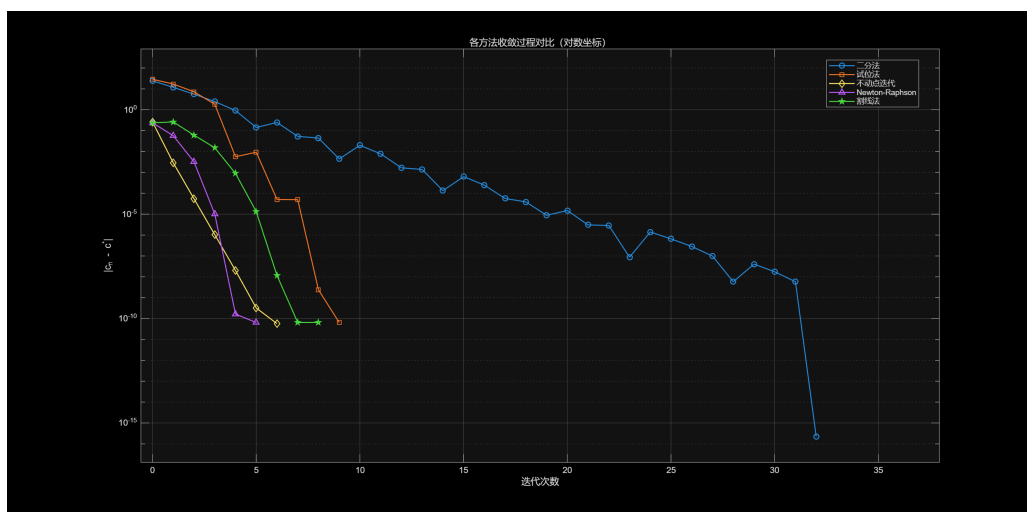


图 2: 各方法收敛过程对比（对数坐标）

收敛性：Newton-Raphson 法和割线法具有最快的收敛速度，分别仅需 5 次和 7 次迭代即达到机器精度。不动点迭代虽然理论上是线性收敛，但因收敛因子 $|g'(c)|$ 较小，实际表现优异（6 次）。二分法收敛最慢（33 次），但严格遵循区间减半规律，行为最可预测。

稳定性：二分法和修正试位法因为始终在有根区间内搜索，所以非常稳定。不动点迭代的收敛性取决于迭代函数的选择，可能需要多次试错来找到合适的迭代函数。Newton-Raphson 法在初始值偏离根较远或函数导数很小时可能发散（如跳到负数区域），需要加入防护机制。割线法同样对初始值敏感，但不需要计算导数。

精度：Newton-Raphson 法和割线法最终达到了机器精度级别（ $|f(c)| \approx 10^{-15}$ ），远优于其他方法。二分法和不动点迭代法可能需要更多的迭代次数来达到同样的精度。

使用条件：二分法和修正试位法需要知道所求根的所在区间；不动点迭代需要构造满足收敛条件（ $|g'(c)| < 1$ ）的迭代函数；Newton-Raphson 法需要导数信息，且对初始值敏感；割线法需要两个初始猜测点，但不需要导数，是 Newton 法的良好替代。

9 附：完整 MATLAB 程序及运行结果

9.1 完整 MATLAB 程序

```
%% 降落伞问题：求解阻力系数 c
clear; clc;

%% 参数设置
g = 9.81;
m = 68.1;
t = 10;
v_target = 30;

tol = 1e-8;          % 收敛容差
max_iter = 100;      % 最大迭代次数

f = @(c) sqrt(g*m./c) .* tanh(sqrt(g*c/m) * t) - v_target;

% 解析导数 f'(c)
alpha = sqrt(g*m);
beta = t*sqrt(g/m);
df_analytical = @(c) alpha * ( ...
    -1./(2*c.^(3/2)) .* tanh(beta*sqrt(c)) ...
    + 1./sqrt(c) .* (1 - tanh(beta*sqrt(c)).^2) .* beta./(2*sqrt(c)) );

% 数值导数（中心差分）
h_diff = 1e-8;
df_numerical = @(c) (f(c + h_diff) - f(c - h_diff)) / (2*h_diff);

% 不动点迭代函数
g_fp = @(c) g*m / v_target^2 * tanh(sqrt(g*c/m) * t)^2;

%% 1. 二分法
[c1, iter1, ~] = my_bisection(f, 0.1, 50, tol, max_iter);
fprintf('二分法: c = %.8f, 迭代次数 = %d\n', c1, iter1);

%% 2. 试位法（修正版）
[c2, iter2, ~] = my_regula_falsi(f, 0.1, 50, tol, max_iter);
fprintf('试位法: c = %.8f, 迭代次数 = %d\n', c2, iter2);

%% 3. 不动点迭代
[c3, iter3, ~] = my_fixed_point(g_fp, 1.0, tol, max_iter);
fprintf('不动点迭代: c = %.8f, 迭代次数 = %d\n', c3, iter3);
```

```

%% 4a. Newton-Raphson (解析导数)
[c4a, iter4a, ~] = my_newton(f, df_analytical, 0.5, tol, max_iter);
fprintf('Newton法(解析导数): c = %.8f, 迭代次数 = %d\n', c4a, iter4a);

%% 4b. Newton-Raphson (数值导数)
[c4b, iter4b, ~] = my_newton(f, df_numerical, 0.5, tol, max_iter);
fprintf('Newton法(数值导数): c = %.8f, 迭代次数 = %d\n', c4b, iter4b);

%% 5. 割线法
[c5, iter5, ~] = my_secant(f, 0.5, 1.0, tol, max_iter);
fprintf('割线法: c = %.8f, 迭代次数 = %d\n', c5, iter5);

%% ===== 局部函数 =====

function [root, iter, history] = my_bisection(f, a, b, tol, max_iter)
    if f(a) * f(b) > 0
        error('二分法: f(a)与f(b)同号, 请重新选取区间。');
    end
    history = [];
    for iter = 1:max_iter
        c = (a + b) / 2;
        history = [history; c];
        if abs(f(c)) < tol || (b - a)/2 < tol
            root = c; return;
        end
        if f(a) * f(c) < 0, b = c; else, a = c; end
    end
    root = (a + b) / 2;
    history = [history; root];
end

function [root, iter, history] = my_regula_falsi(f, a, b, tol, max_iter)
    fa = f(a); fb = f(b);
    if fa * fb > 0
        error('试位法: f(a)与f(b)同号。');
    end
    history = []; stag_a = 0; stag_b = 0;
    for iter = 1:max_iter
        c = b - fb * (b - a) / (fb - fa);
        fc = f(c);
        history = [history; c];
        if abs(fc) < tol, root = c; return; end
        if fa * fc < 0
            b = c; fb = fc; stag_b = 0;
            stag_a = stag_a + 1;
        end
    end
end

```

```

        if stag_a >= 2, fa = fa / 2; end
    else
        a = c; fa = fc; stag_a = 0;
        stag_b = stag_b + 1;
        if stag_b >= 2, fb = fb / 2; end
    end
end
root = c;
end

function [root, iter, history] = my_fixed_point(g, x0, tol, max_iter)
    x = x0; history = [x];
    for iter = 1:max_iter
        x_new = g(x);
        history = [history; x_new];
        if abs(x_new - x) < tol, root = x_new; return; end
        if abs(x_new) > 1e10 || isnan(x_new) || ~isreal(x_new)
            warning('不动点迭代发散。');
            root = x_new; return;
        end
        x = x_new;
    end
    root = x;
end

function [root, iter, history] = my_newton(f, df, x0, tol, max_iter)
    x = x0; history = [x];
    for iter = 1:max_iter
        f_val = f(x); df_val = df(x);
        if abs(df_val) < 1e-15
            warning('Newton: 导数接近零。');
            root = x; return;
        end
        x_new = x - f_val / df_val;
        history = [history; x_new];
        if x_new <= 0
            x_new = x / 2;
            history(end) = x_new;
        end
        if abs(x_new - x) < tol, root = x_new; return; end
        x = x_new;
    end
    root = x;
end

```

```
function [root, iter, history] = my_secant(f, x0, x1, tol, max_iter)
    history = [x0; x1];
    for iter = 1:max_iter
        f0 = f(x0); f1 = f(x1);
        if abs(f1 - f0) < 1e-15
            warning('割线法：分母接近零。');
            root = x1; return;
        end
        x2 = x1 - f1 * (x1 - x0) / (f1 - f0);
        history = [history; x2];
        if x2 <= 0, x2 = x1 / 2; history(end) = x2; end
        if abs(x2 - x1) < tol, root = x2; return; end
        x0 = x1; x1 = x2;
    end
    root = x1;
end
```

9.2 完整运行结果

导数验证：

c = 0.5: 解析 = -34.39970613, 数值 = -34.39970619, 差异 = 6.61e-08

c = 1.0: 解析 = -12.81140054, 数值 = -12.81140065, 差异 = 1.16e-07

二分法: c = 0.73793065, 迭代次数 = 33, f(c) = -1.29e-09

试位法(修正): c = 0.73793065, 迭代次数 = 10, f(c) = -2.36e-12

不动点迭代: c = 0.73793065, 迭代次数 = 6, f(c) = -1.48e-10

Newton法(解析导数): c = 0.73793065, 迭代次数 = 5, f(c) = 3.55e-15

Newton法(数值导数): c = 0.73793065, 迭代次数 = 5, f(c) = 0.00e+00

割线法: c = 0.73793065, 迭代次数 = 7, f(c) = 3.55e-15